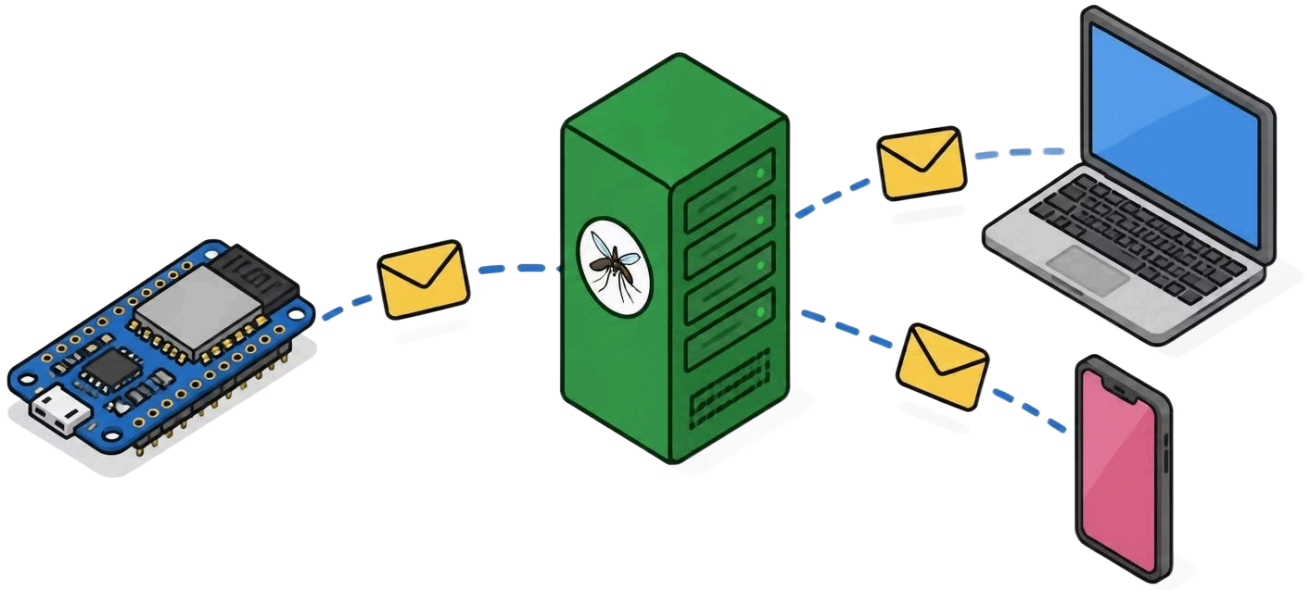
Fundamentos de Sistemas Embebidos

MQTT y Mosquitto: mensajería para dispositivos



El problema

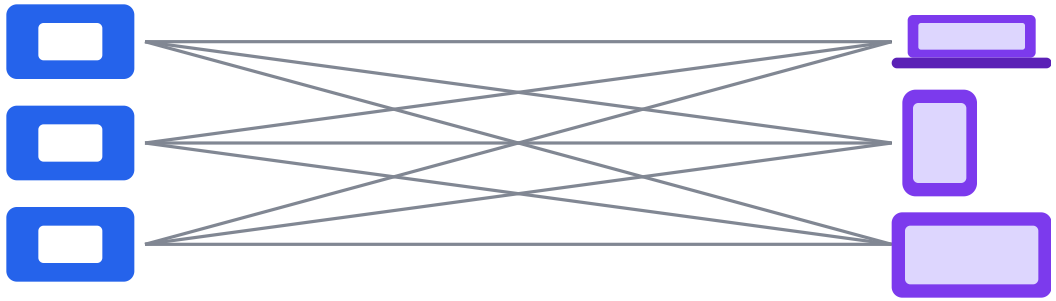
Conectar todo con todo no escala

Un ESP32 mide temperatura; varios programas necesitan el dato.

Conexiones directas

9

3 emisores × 3 receptores



- Agregar receptores obliga a modificar al emisor.
- Si uno se apaga, el emisor debe enterarse.

Con broker

6

Los mismos seis clientes conectados al centro



- Emisor y receptores no se conocen entre sí.
- Las conexiones dejan de multiplicarse entre todos.

Aplicación ≠ transporte ≠ enlace

MQTT en el mapa de capas

Capa	Protocolos y tecnologías
Aplicación	**MQTT**, HTTP, WebSocket, DNS, DHCP, tu protocolo
Transporte	TCP · UDP para DNS
Red	IP
Enlace	Wi-Fi / Ethernet

MQTT no compite con TCP: lo usa.

- HTTP, WebSocket y MQTT son protocolos de aplicación.
- Los tres viajan sobre TCP.
- MQTT usa cualquier enlace con IP.
- ESP-NOW, BLE, LoRa y RF transparente no llevan IP.
- Sin IP, se llega por una puerta de enlace.

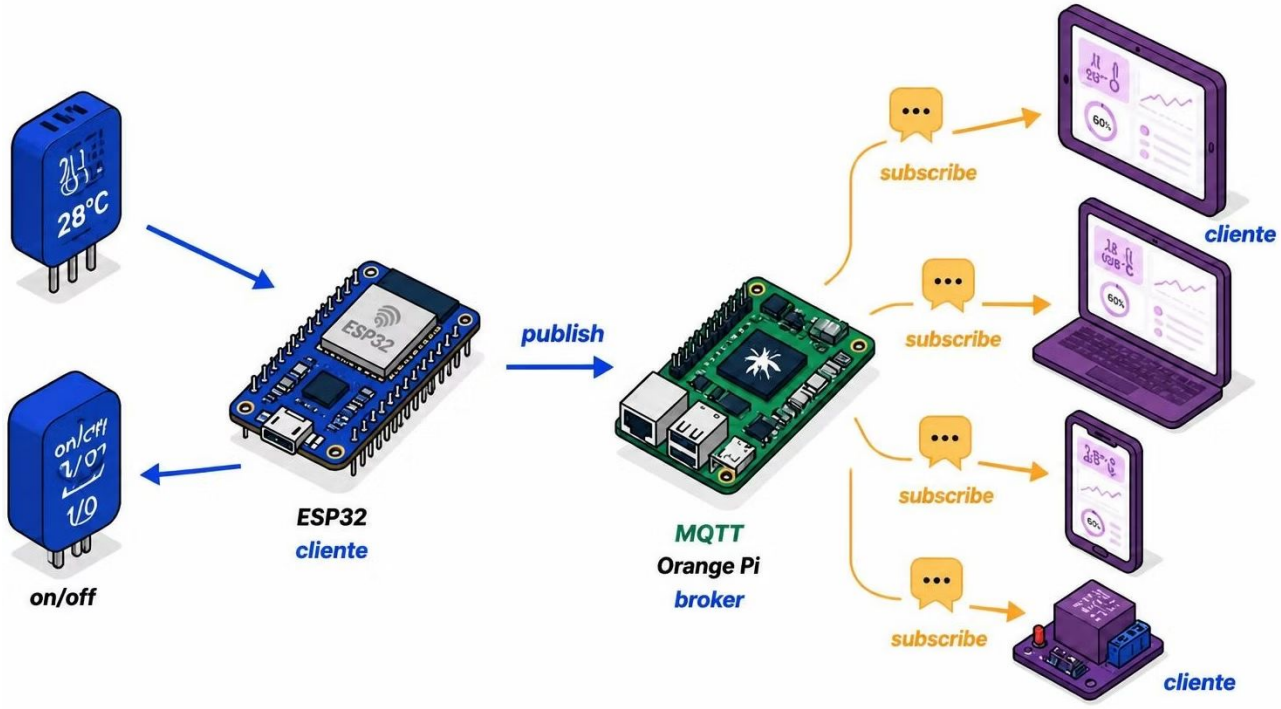
`Sin IP → puente → MQTT`

Qué es MQTT

- Protocolo de mensajes de tipo publicar / suscribirse.
- Emisor y receptor solo conocen al broker.
- Pensado para dispositivos pequeños y redes limitadas.
- Funciona sobre TCP/IP; puerto usual 1883.

Conceptos clave y analogía del chat

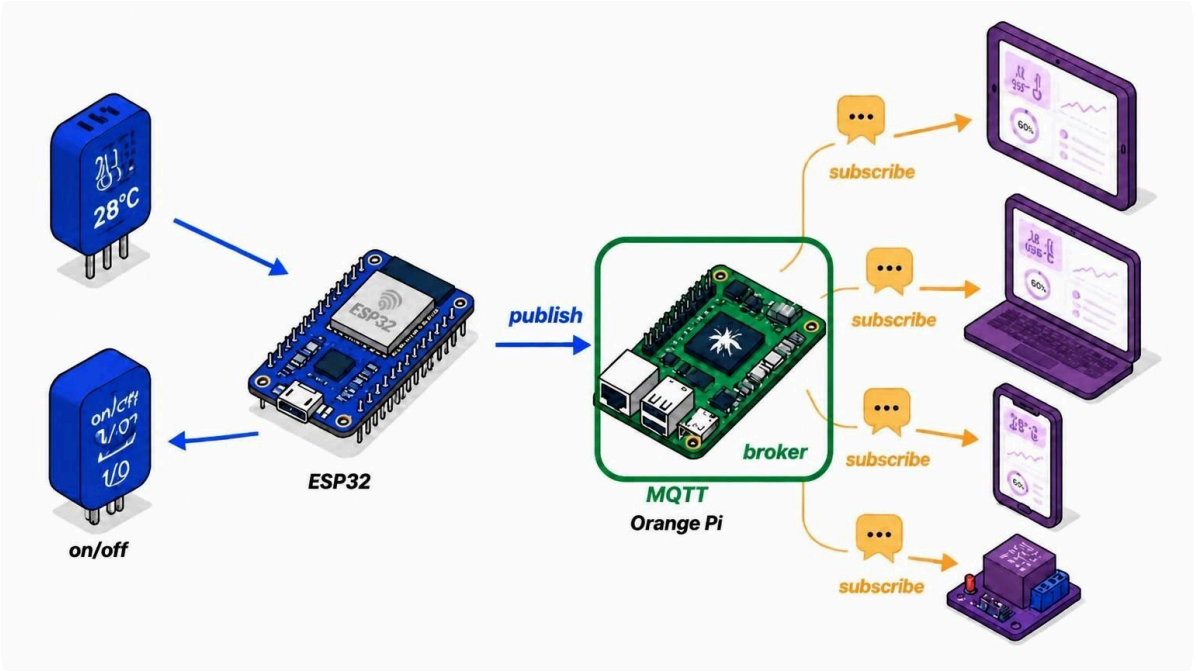
En el chat	En MQTT	Qué significa
Servidor de la aplicación	**Broker**	Servidor central: Mosquitto. Intermedia entre publicadores y suscriptores.
Persona con su teléfono	**Cliente**	Programa o dispositivo: ESP32, Node-RED o tu PC.
Grupo de chat	**Tema (topic)**	Nombre del canal: casa/sala/temperatura.
Escribir en el grupo	**Publicar**	Mandar un mensaje a un tema.
Unirse al grupo	**Suscribirse**	Pedir al broker los mensajes de un tema.
Texto del mensaje	**Payload**	Contenido del mensaje que viaja por el tema.



El centro de la estrella

¿Quién administra la comunicación y dónde está el broker?

- MQTT es una red en **estrella**: el centro es el broker.
- Cada cliente tiene una sola conexión, hacia el broker.
- El broker reparte mensajes, valida usuarios y guarda retenidos.
- Los clientes no se conocen: desacoplados, pero con un centro.
- El broker es un **programa**, no un aparato.
- En el curso corre en la Orange Pi; se ubica por IP y puerto `1883`.



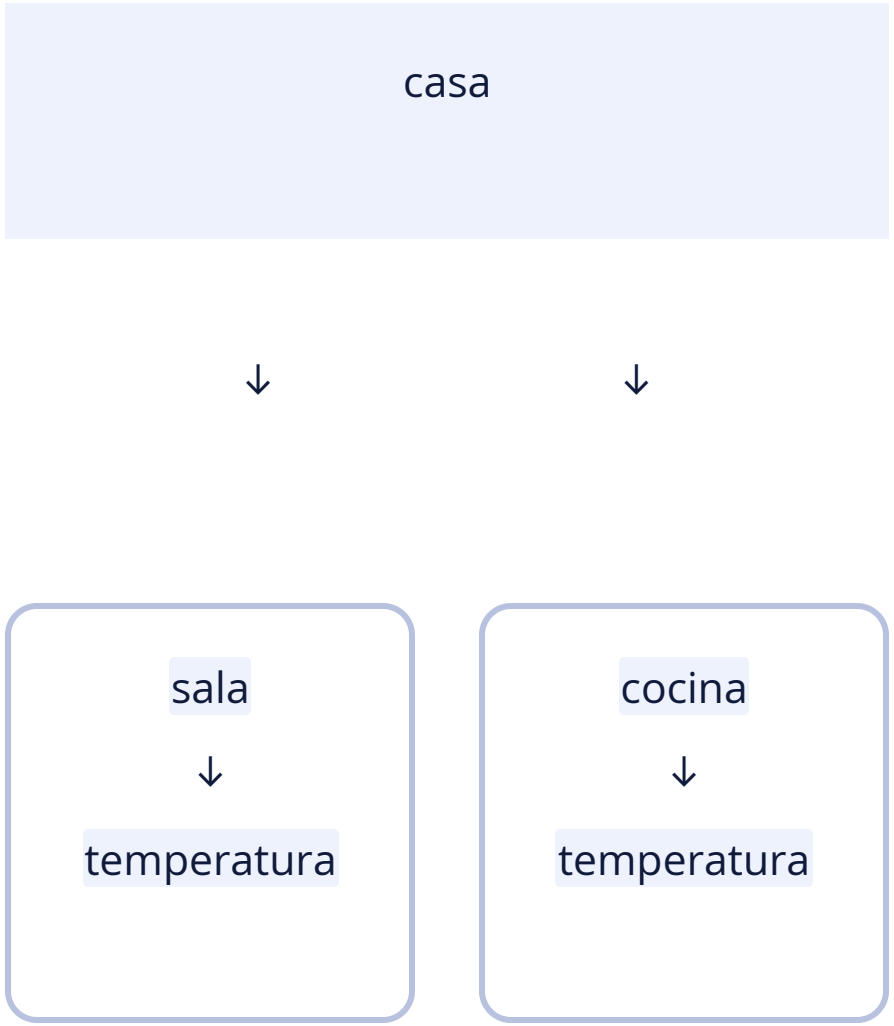
Red	Inteligencia	Si falla el centro
MQTT con broker	En el broker	Se detienen los mensajes
ESP-NOW, Zigbee y Thread (malla)	En los nodos	La red se reorganiza
Wi-Fi con router	En el router	Se cae la red

Canales con niveles

Temas y comodines

```
casa/sala/temperatura
casa/cocina/temperatura
casa/+/temperatura
casa/#
proyecto/esp32-01/estado
proyecto/esp32-01/orden/led
```

- Los temas no se registran: existen cuando alguien publica.
- `+` sustituye un nivel; `#` todo lo que cuelga.
- Los comodines solo se usan al suscribirse, nunca al publicar.



El mensaje

Publish, Subscribe y Payload

ESP32

Publish →

Envía al tema

Tema

casa/sala/temperatura

Payload

22.5

Panel

← Subscribe

Escucha el tema

- El tema dice dónde va; el payload, qué se envía.
- Un cliente puede publicar y suscribirse a distintos temas.
- Payload corto; JSON para varios datos juntos.

Texto, JSON o binario

```
hola
22.5
{"temperatura": 22.5, "humedad": 60}
```

QoS y mensajes retenidos

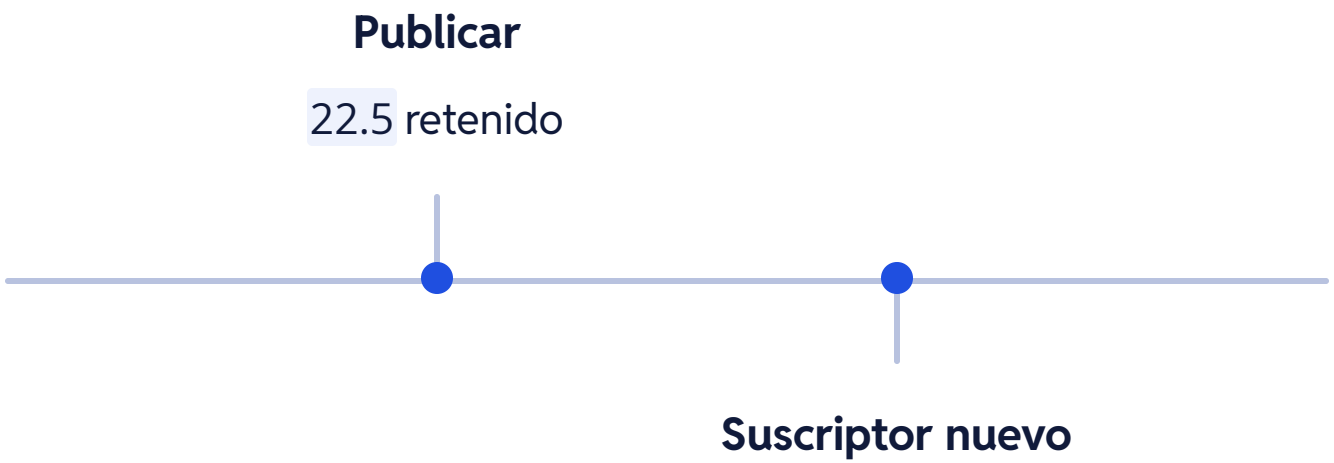
Más garantía implica más mensajes de control.

Nivel	Garantía	Control de entrega
QoS 0	Como máximo una vez	Sin confirmación; lecturas periódicas
QoS 1	Al menos una vez	Con confirmación; puede repetirse; órdenes
QoS 2	Exactamente una vez	Doble confirmación

Retenido

El broker guarda el último mensaje del tema.

Útil para estados; carpeta `data`.



MQTT contra HTTP y WebSocket

Un panel quiere mostrar la temperatura de un ESP32.

Criterio	HTTP	WebSocket	MQTT
Quién inicia	Solo el cliente	Cualquiera, canal abierto	Cualquiera publica cuando hay dato
Dato nuevo	Polling: preguntar y preguntar	Llega por el canal	Suscripción única; el broker avisa
Muchos receptores	Una petición por cada uno	Una conexión con cada uno	ESP32 publica una vez; broker reparte
ESP32	Servidor	Servidor	Cliente; una conexión al broker
Garantía	La de TCP	La de TCP	QoS 0, 1 o 2
Caída / último valor	Sin mecanismo estándar	Lo resuelve tu programa	`keep alive`, Last Will y retenido

HTTP sirve páginas (`/html`). WebSocket da el canal; MQTT agrega reglas.

Mosquitto abre `8080` con `protocol websockets`.

Qué programas tú y qué resuelve el protocolo

MQTT contra TCP y UDP con sockets propios

Caso	Sockets propios	MQTT
Máxima rapidez a un destino	UDP: ~1633 envíos/s medidos; STA, con manejo de núcleos	No busca tasa máxima; sin medir: por confirmar
Orden que no debe perderse	TCP; o UDP con confirmación y reintento propios	QoS 1 incluye confirmación
Varios receptores	UDP broadcast, misma red	Suscritos, aun en otras redes
Cambio de IP del receptor	Reconfigurar al emisor	Solo conoce al broker
Agregar receptor	Otro socket o cambiar emisor	Se suscribe y listo

Sockets propios: direcciones, confirmaciones y reintentos a tu cargo.

MQTT lo resuelve a cambio de depender del broker.

MQTT y las tecnologías inalámbricas

Tecnología	Qué resuelve	Relación con MQTT
Wi-Fi	Pila IP; alto ancho de banda y consumo	MQTT directo sobre él; ESP32-S3
ESP-NOW	ESP32 con ESP32; sin router ni IP; ~250 bytes	ESP32 «puerta» recibe y publica
BLE	Teléfono cercano; muy bajo consumo	Teléfono o puerta de enlace publica
LoRa / LoRaWAN	Kilómetros; cientos de bps a decenas de kbps	Depende de la plataforma: por confirmar
Zigbee / Thread	Malla de bajo consumo para domótica	Zigbee2MQTT convierte Zigbee en MQTT
RF transparente (HC-12)	Serial por aire; sin direcciones ni confirmación	El microcontrolador hace el puente

MQTT organiza mensajes; no sustituye al enlace.

¿Qué uso en cada caso?

Necesidad	Opción	Por qué
Servir una página o archivo	HTTP	Recurso completo
Datos en vivo: ESP32 a navegador	WebSocket	Canal permanente directo
Audio o envío muy rápido entre dos puntos	UDP	Mínima latencia; dato reemplazable
ESP32 cercanos, sin router	ESP-NOW	Sin infraestructura; arranque inmediato
Teléfono cercano, nodo con pila	BLE	Muy bajo consumo
Kilómetros con pila	LoRa	Alcance por velocidad
Orden que no debe perderse	TCP / MQTT QoS 1	Entrega confirmada
Muchos dispositivos y receptores; guardar y mostrar datos	MQTT	Desacopla emisores y receptores

MQTT no conviene para audio/video continuo, páginas, ausencia de IP o pilas por meses. Si cae el broker, no circulan mensajes.

De los enlaces remotos al sistema embebido

Origen y dónde se usa

IBM y Arcom lo crearon para vigilar oleoductos por satélite (SCADA).

Finales de los 90

Compacto, bajo consumo y fácil de implementar

Hacia 2008

Aparece Mosquitto

2014

Estándar OASIS

2016

ISO/IEC 20922

2019

MQTT 5

Hoy

Industria · vehículos · agricultura · salud · hogar inteligente

La carga de repartir queda en el broker

Por qué encaja en un sistema embebido

2

bytes, desde

Paquete fijo compacto.

Diseñado para enlaces caros y poca energía, como el satelital.

Una conexión

El dispositivo abre una conexión hacia el broker.

El broker atiende a los receptores; el ESP32 no es servidor.

Control por mensaje

QoS 0, 1 o 2.

Desconexiones detectadas con keep alive y Last Will.

Un ESP32 como cliente MQTT

ESP-IDF incluye ESP-MQTT, con MQTT v5.0. Ejemplos: `protocols/mqtt` y `protocols/mqtt5`.

Esquema ilustrativo. Confirmar nombres contra ESP-IDF v6.0.2 antes de usarlo.

```
esp_mqtt_client_config_t cfg = {
    .broker.address.uri = "mqtt://<IP-de-la-placa>:1883",
    .credentials.username = "<usuario>",
    .credentials.authentication.password = "<contraseña>",
    .session.last_will.topic = "casa/sala/estado",
    .session.last_will.msg = "offline",
    .session.last_will.qos = 1,
    .session.last_will.retain = 1,
};
esp_mqtt_client_handle_t client = esp_mqtt_client_init(&cfg);
esp_mqtt_client_start(client);

// Una vez conectado (evento MQTT_EVENT_CONNECTED):
esp_mqtt_client_publish(client, "casa/sala/temperatura", "22.5", 0, 0, 0);
```

URI `mqtt://` o `mqtts://` (TLS). Usuario, contraseña y Last Will en la configuración.

Saber si el dispositivo está en línea

- *Keep alive*: avisa cada cierto tiempo que sigue conectado.
- Si deja de avisar, el broker lo considera desconectado.
- *Last Will*: el broker publica un mensaje si el cliente cae.
- **Retenido** guarda el último estado.
- MQTT 5 agrega retardo del Last Will contra falsas alarmas.

Momento	Publica	Tema	Payload	Ret.
Al conectarse	ESP32	`casa/sala/estado`	`online`	Sí
Caída sin aviso	Broker: Last Will	`casa/sala/estado`	`offline`	Sí



El mismo protocolo, distintos entornos

Ecosistema y seguridad

Home Assistant · Zigbee2MQTT · Tasmota · ESPHome · Node-RED

Se conectan al broker. AWS IoT Core habla MQTT con QoS 0 y 1.

Red local

1883

Nuestro broker pide usuario y contraseña, pero no cifra.

Internet

8883

TLS; puerto usual. En ESP-MQTT: `mqtts://`.

- Un usuario por dispositivo; revocar solo el comprometido.
- Mosquitto limita temas por usuario: `acl_file`, por confirmar.

De los conceptos a la práctica

Resumen y lo que sigue

- MQTT desacopla a quien manda y quien recibe mediante un broker.
- Mensajes a un tema; QoS fija la garantía; retenido guarda el último valor.
- MQTT conviene con muchos dispositivos y receptores; si no, hay otras opciones.
- Mosquitto es el broker que instalaremos en la Orange Pi, con Docker.
- Arquitectura: ESP32 publica, Mosquitto reparte, Node-RED procesa.
- En la práctica: Portainer, Mosquitto y pruebas con usuario y contraseña.

